

A Comparative Study of Machine Learning and Deep Learning Approaches for Remaining Useful Life Prediction in Turbofan Engines Using NASA C-MAPSS

Sebastian Martinez Abarca
LEAD University

BCD-6210 Advanced Data Mining – IC 2026
Email: sebastian.martinez@ulead.ac.cr

Dr. Juan Murillo-Morera
Universidad LEAD

Course Professor
Email: jmurillo@ulead.ac.cr

Abstract—This paper presents a comparative study of machine learning (ML) and deep learning (DL) approaches for predicting Remaining Useful Life (RUL) of turbofan engines using the NASA C-MAPSS dataset. We evaluate eight models—Linear Regression, SVR, Random Forest, XGBoost, LSTM, GRU, TCN, and Transformer—across four sub-datasets (FD001–FD004) with increasing operational complexity. A dual-framing approach addresses RUL as both regression and binary classification. Optuna optimization (800 trials) and ensemble averaging enhance performance. SHAP analysis identifies sensor_11 (HPC outlet temperature) as the dominant degradation indicator. The DL ensemble achieves RMSE of 11.95 and R^2 of 0.91, outperforming classical ML by 33–59%. An interactive web application enables real-time prediction and model comparison.

Index Terms—Predictive Maintenance, Remaining Useful Life, C-MAPSS, LSTM, GRU, Transformer, XGBoost, SHAP, Deep Learning, Ensemble Learning, Optuna, Turbofan Engine

I. INTRODUCTION

The aviation industry demands the highest standards of reliability for its propulsion systems. Turbofan jet engines are subject to progressive degradation through thermal cycling, mechanical wear, and environmental exposure [1]. Unplanned engine failures carry catastrophic consequences in terms of safety and economic losses [3].

Prognostics and Health Management (PHM) leverages sensor data and computational models to predict equipment degradation before failure. At its core lies Remaining Useful Life (RUL): the estimated number of operational cycles before an engine requires maintenance [2]. Condition-based maintenance, enabled by accurate RUL prediction, performs maintenance precisely when needed, reducing both unplanned downtime and unnecessary inspections [6].

NASA’s C-MAPSS dataset has become the gold standard for evaluating RUL methodologies, providing run-to-failure data under varying operational conditions and fault modes [1].

This study addresses four research questions:

- **RQ1:** How do classical ML models compare with DL architectures in RUL prediction accuracy?

- **RQ2:** How does operational complexity impact predictive performance?
- **RQ3:** Which sensor features are most relevant for degradation detection?
- **RQ4:** Can RUL be effectively reframed as binary classification for practical failure alerting?

II. BACKGROUND

A. Remaining Useful Life

For an engine at cycle t with total lifetime T , we adopt a piece-wise linear degradation model with cap $R_{max} = 125$ cycles [2]:

$$RUL_{pw}(t) = \min(T - t, R_{max}) \quad (1)$$

B. NASA Scoring Function

The PHM community employs an asymmetric scoring function that penalizes late predictions more severely [1]:

$$S = \sum_{i=1}^N s_i, \quad s_i = \begin{cases} e^{-d_i/13} - 1, & d_i < 0 \\ e^{d_i/10} - 1, & d_i \geq 0 \end{cases} \quad (2)$$

where $d_i = \hat{RUL}_i - RUL_i$. This asymmetry reflects the operational reality: underestimating RUL means missing a maintenance window.

C. Model Families

Classical ML: Linear Regression provides the simplest baseline. SVR with RBF kernel captures non-linear relationships. Random Forest constructs an ensemble of decision trees. XGBoost implements gradient-boosted trees with L1/L2 regularization [9].

Deep Learning: LSTM [10] networks learn long-term temporal dependencies through gating mechanisms. GRU [11] simplifies LSTM with a unified update gate. TCN [12] applies dilated causal convolutions for large receptive fields without deep stacking. Transformers [13] use multi-head self-attention to model dependencies between any two positions in a sequence.

III. RELATED WORK

Saxena et al. [1] established initial baselines (RMSE 30–50 on FD001) using similarity-based methods. Heimes [2] first applied recurrent networks to C-MAPSS. Li et al. [4] achieved RMSE of 12.61 on FD001 with deep CNNs. Zheng et al. [5] proposed LSTM architectures capturing temporal degradation patterns. More recent work explored Transformer-based models [14], hybrid CNN-LSTM hybrids [15], and automated hyperparameter tuning [16].

Our work distinguishes itself by evaluating eight models across four algorithmic families on all sub-datasets, combining Optuna optimization (800 trials), ensemble averaging, SHAP interpretability, and production deployment in a single integrated study.

IV. EXPERIMENTAL DESIGN

A. Dataset Description

Table I summarizes the four sub-datasets with progressively increasing operational complexity.

TABLE I: C-MAPSS Sub-dataset Characteristics

Sub-dataset	Train	Test	Op. Cond.	Fault Modes
FD001	100	100	1	1 (HPC)
FD002	260	259	6	1 (HPC)
FD003	100	100	1	2 (HPC+Fan)
FD004	248	249	6	2 (HPC+Fan)

Each unit provides 26 measurements per cycle: unit identifier, cycle number, 3 operational settings, and 21 sensor readings.

B. Exploratory Data Analysis

Comprehensive EDA revealed the following key findings. The dataset contains zero null values across all sub-datasets. Engine lifetimes range from 128 to 362 cycles in FD001 and 128 to 543 cycles in FD004, reflecting the broader variability introduced by multiple operating conditions and fault modes. The true RUL distribution in the test sets spans 6–195 cycles, with FD004 exhibiting the widest spread and highest mean (86.6 cycles).

Variance analysis across all sensors identified 7 constant or near-constant features (sensors 1, 5, 6, 10, 16, 18, 19 and operational setting 3), which were removed prior to modeling. The remaining 14 sensors showed meaningful variation and progressive degradation patterns.

Figure 1 illustrates the distribution of sensor readings in FD001, clearly distinguishing informative from constant sensors. Degradation analysis, shown in Figure 2, revealed that sensors deviate from their healthy-state baselines primarily during the final 30% of engine life, with the acceleration becoming pronounced in the last 50 cycles. This justifies the piece-wise linear RUL cap at 125 cycles.

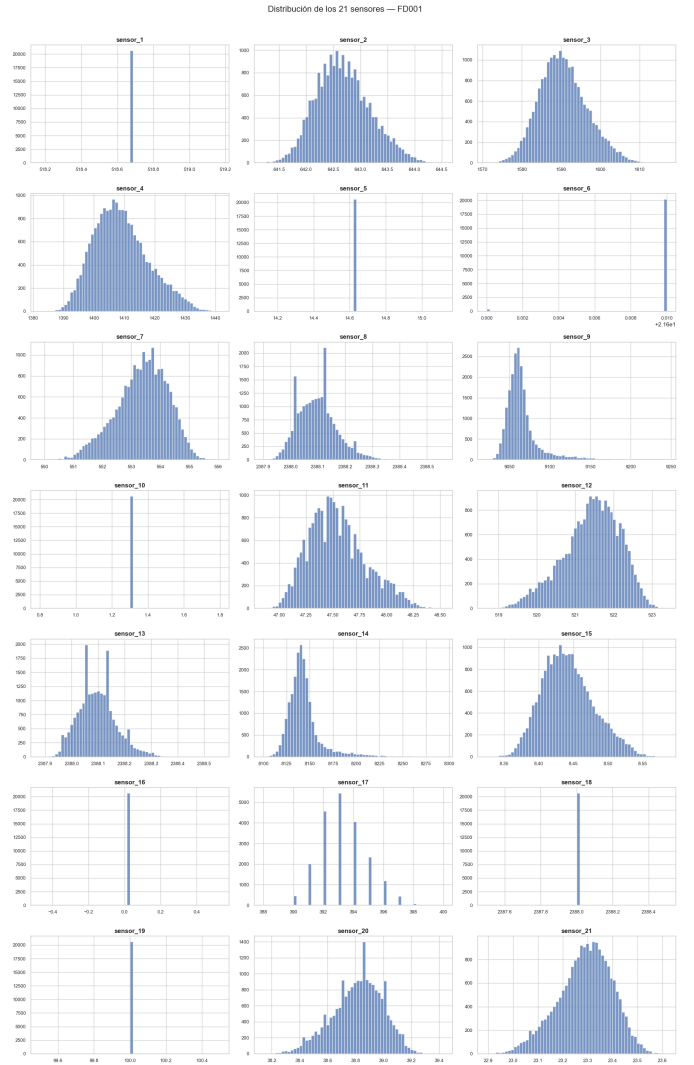


Fig. 1: Distribution of the 21 raw sensor readings in FD001. Seven sensors (shaded) exhibit near-zero variance and are removed prior to modeling.

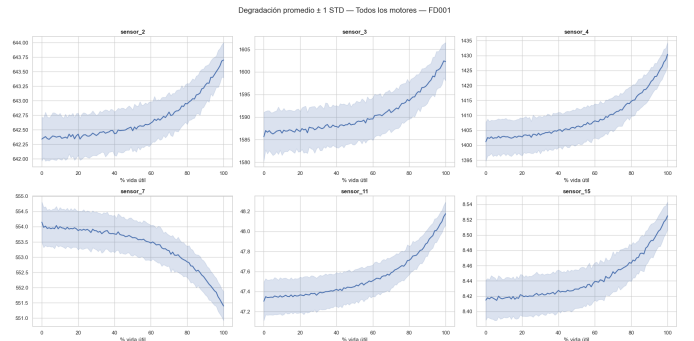


Fig. 2: Mean degradation curves with ± 1 standard deviation for key sensors across engine lifetime percentage. Deviation from healthy state accelerates in the final 30% of life, justifying the RUL cap at 125 cycles.

Correlation analysis identified sensor_11 (HPC outlet tem-

perature) as the feature most linearly correlated with RUL ($r = -0.78$ in FD001), followed by sensor_9 and sensor_4. Notably, these correlations intensify significantly during the final 25% of engine life, consistent with accelerating degradation dynamics. Figure 3 presents the full correlation structure, confirming strong co-degradation between temperature and pressure sensors.

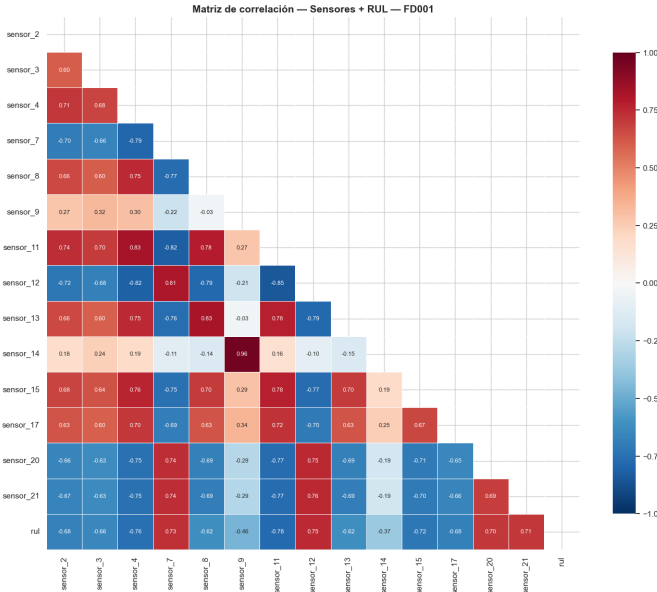


Fig. 3: Correlation matrix of useful sensors and RUL in FD001. Strong negative correlations between temperature sensors (sensor_11, sensor_4) and RUL indicate progressive thermal degradation in the HPC.

For the binary classification framing, using a threshold of $W = 30$ cycles, the positive class (failure within 30 cycles) represents 12.5–15% of samples across sub-datasets, indicating moderate but manageable class imbalance. Figure 4 summarizes the RUL distribution in the test sets and the piecewise linear target construction.

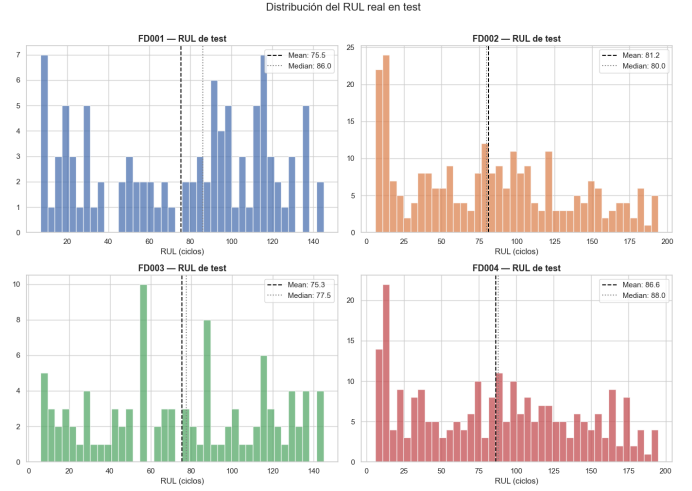


Fig. 4: Distribution of true RUL values in the test sets across all four sub-datasets. FD004 shows the widest spread, reflecting higher operational complexity.

C. Preprocessing Pipeline

The preprocessing pipeline consists of five stages:

- 1) RUL Computation:** $RUL_j(t) = \min(T_j - t, 125)$.
- 2) Binary Label Generation:** $y_j(t) = \mathcal{K}[RUL_j(t) \leq 30]$.
- 3) Constant Feature Removal:** 7 sensors with variance < 0.001 are removed, reducing to 14 useful features.
- 4) Operating Condition Normalization:** For FD002 and FD004, sensor readings are z-score normalized within each operating condition cluster, removing the confounding effect of different operating regimes.
- 5) Min-Max Scaling:** All features are scaled to $[0, 1]$ fitted on training data only to prevent data leakage.

D. Feature Engineering

Rolling window statistics ($w = 30$ cycles per engine) expand the feature space from 16 to 58 features:

- Rolling mean: $\bar{x}_{s,t} = \frac{1}{w} \sum_{i=t-w+1}^t x_{s,i}$
- Rolling std: $\sigma_{s,t} = \sqrt{\frac{1}{w-1} \sum_i (x_{s,i} - \bar{x}_{s,t})^2}$
- Trend: $\Delta_{s,t} = \bar{x}_{s,t} - \bar{x}_{s,t-1}$

E. Model Configurations

1) *Classical ML:* Classical models receive the 16 base features (test set contains only the final cycle per engine, so rolling features are unavailable).

TABLE II: Classical ML Model Configurations

Model	Key Hyperparameters
Linear Regression	Default
SVR	RBF kernel, $C = 100$, $\epsilon = 0.1$
Random Forest	200 trees
XGBoost	300 trees, depth=6, lr=0.05

2) *Deep Learning*: DL models process sequences of $L = 80$ cycles with all 58 features. The RUL target is normalized to $[0, 1]$ by dividing by 125. All models use AdamW with weight decay 10^{-4} , early stopping (patience=10), and ReduceLROnPlateau. Each architecture ends with BatchNorm, a 48-unit dense layer (ReLU, Dropout 0.3), and a single output neuron.

F. Hyperparameter Optimization

Optuna [16] with median pruning optimized 16 model-dataset combinations (4 models $\times$ 4 datasets), 50 trials each, for 800 total runs. The search space covered hidden size $\{64, 128, 192, 256\}$, layers $\{1, 2, 3\}$, dropout $[0.2, 0.5]$, learning rate $[10^{-4}, 5 \times 10^{-3}]$, weight decay $[10^{-5}, 10^{-3}]$, batch size $\{32, 64, 128\}$, and Transformer-specific $d_{model} \in \{32, 64, 128\}$, $n_{heads} \in \{2, 4\}$.

G. Ensemble Strategy

Ensemble predictions average all four optimized DL models per dataset:

$$R\hat{U}L_{ens} = \frac{1}{4} \sum_{m=1}^4 R\hat{U}L_m \quad (3)$$

V. ANALYSIS AND RESULTS

A. Summary of Key Findings

Before presenting individual results, we summarize the main outcomes of this study. Classical ML models achieve acceptable performance on single-condition datasets but degrade substantially under multi-condition scenarios. Deep learning architectures consistently outperform classical ML by 33–59% RMSE reduction. The DL ensemble achieves the best overall performance, with $RMSE \leq 13.5$ on three of four sub-datasets and $R^2 \geq 0.90$. Hyperparameter optimization revealed strong dataset-specificity: optimal configurations differ substantially between sub-datasets, confirming that a universal architecture is suboptimal. SHAP analysis consistently identified sensor_11 and sensor_4 as the dominant degradation indicators across all sub-datasets and both model families. Binary classification reached AUC-ROC of 0.983, demonstrating practical viability for failure alerting systems.

B. Classical ML Results

TABLE III: Classical ML Regression Results — FD001

Model	RMSE	MAE	R ²	NASA Score
Linear Regression	21.91	17.61	0.722	1318.9
SVR (RBF)	19.61	13.82	0.777	1624.9
Random Forest	17.92	12.92	0.814	886.4
XGBoost	18.31	13.39	0.806	1058.1

Random Forest achieves the best performance on FD001 (RMSE 17.92, R^2 0.814). SVR shows a disproportionately high NASA Score (1624.9) despite moderate RMSE, indicating a systematic tendency toward late predictions—operationally the most dangerous failure mode.

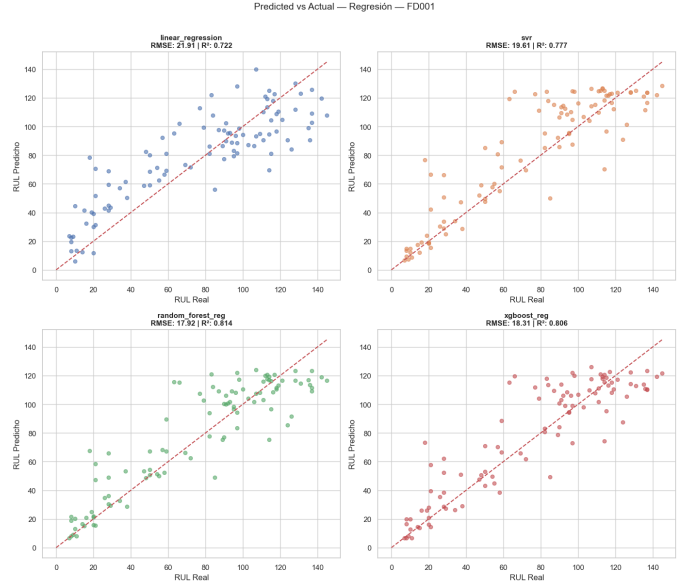


Fig. 5: Predicted vs. actual RUL for classical ML models on FD001. The diagonal represents perfect prediction. Random Forest shows the tightest clustering around the ideal line.

Cross-dataset results (Table IV) reveal two consistent patterns: performance degrades 50–180% from single-condition to multi-condition datasets, and XGBoost outperforms Random Forest on multi-condition scenarios (FD002, FD004), suggesting gradient boosting handles operational heterogeneity more effectively.

TABLE IV: Cross-Dataset RMSE — Classical ML

Model	FD001	FD002	FD003	FD004
Linear Reg.	21.91	32.50	22.87	36.62
SVR	19.61	51.77	22.23	55.46
Random Forest	17.92	29.29	21.24	31.09
XGBoost	18.31	28.92	21.44	30.83

C. Binary Classification Results

TABLE V: Binary Classification Results — FD001 ($W = 30$ cycles)

Model	Acc.	Prec.	Rec.	F1	AUC
Random Forest	0.91	0.90	0.72	0.80	0.983
XGBoost	0.91	0.90	0.72	0.80	0.980

Both classifiers achieve excellent AUC-ROC (≥ 0.98). However, recall of 0.72 indicates that 28% of engines approaching failure are not flagged at the default threshold—a concern for safety-critical deployment that motivates threshold tuning toward higher W values.

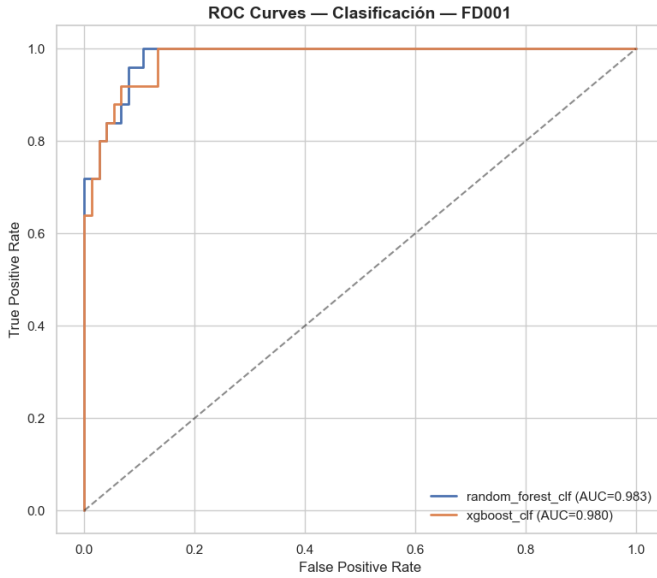


Fig. 6: ROC curves for binary classification. Both classifiers achieve $AUC \geq 0.98$, confirming strong discriminative ability for failure alerting.

D. Deep Learning Results

TABLE VI: Deep Learning Regression Results — FD001

Model	RMSE	R^2	NASA Score
LSTM	14.24	0.873	5751.7
GRU	14.89	0.861	9900.3
TCN	14.39	0.870	10400.8
Transformer	13.16	0.892	7681.2
Ensemble	11.97	0.910	—

The Transformer achieves the best individual result on FD001 (RMSE 13.16, R^2 0.892), likely benefiting from its ability to attend to critical degradation time steps regardless of sequence position. Ensemble averaging yields an additional 9% RMSE improvement, reducing it to 11.97 with $R^2 = 0.910$.

TABLE VII: Cross-Dataset RMSE / R^2 — DL (Optuna-Optimized)

Model	FD001	FD002	FD003	FD004
LSTM	14.24/0.87	12.80/0.89	15.56/0.86	17.92/0.82
GRU	14.89/0.86	14.86/0.86	13.71/0.89	18.66/0.81
TCN	14.39/0.87	13.43/0.88	16.82/0.84	21.48/0.75
Transformer	13.16/0.89	14.28/0.87	13.34/0.90	17.77/0.83
Ens.	11.97/0.91	11.95/0.91	13.46/0.90	17.32/0.84

The Transformer shows the most consistent individual performance (R^2 range: 0.83–0.90). LSTM excels on FD002 (RMSE 12.80), while TCN exhibits the largest degradation on FD004 (RMSE 21.48), suggesting sensitivity to combined operational and fault complexity. The ensemble consistently outperforms all individual models by 1–3 RMSE points across all sub-datasets.

E. ML vs. DL Comparison

TABLE VIII: Best Classical ML vs. DL Ensemble

Dataset	Best ML	ML RMSE	Ens. RMSE	Improvement
FD001	RF	17.92	11.97	33.2%
FD002	XGB	28.92	11.95	58.7%
FD003	RF	21.24	13.46	36.6%
FD004	XGB	30.83	17.32	43.8%

The DL ensemble consistently outperforms classical ML by 33–59%, with the largest improvement on FD002 (58.7%). This gap reflects the fundamental architectural difference: DL models process temporal sequences of 80 cycles with 58 engineered features, while classical models operate on 16 base features from a single time step.

F. SHAP Interpretability Analysis

SHAP analysis was applied to XGBoost and Random Forest on all four sub-datasets. Figure 7 presents the summary plot for XGBoost on FD001, and Figure 8 shows the cross-dataset importance comparison.

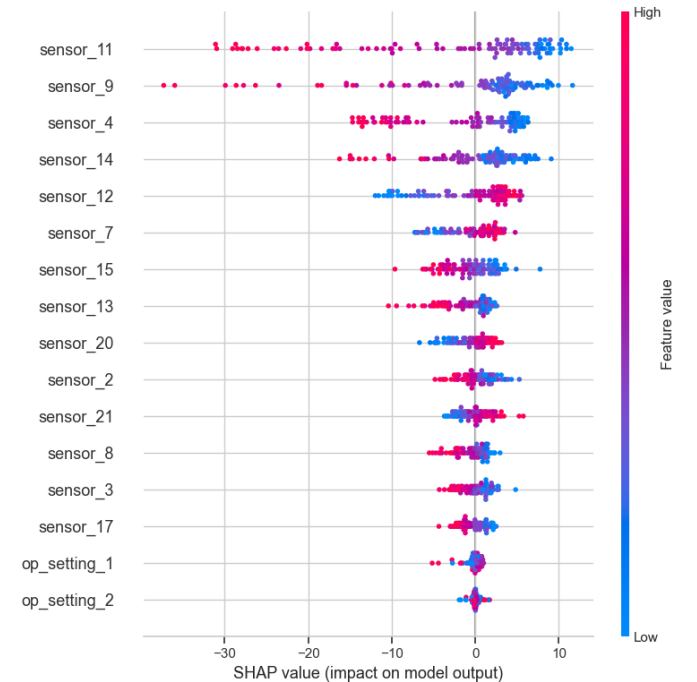


Fig. 7: SHAP summary plot for XGBoost on FD001. Each point represents one test engine; color indicates feature value (red = high, blue = low). Sensor_11 dominates with the widest impact range.

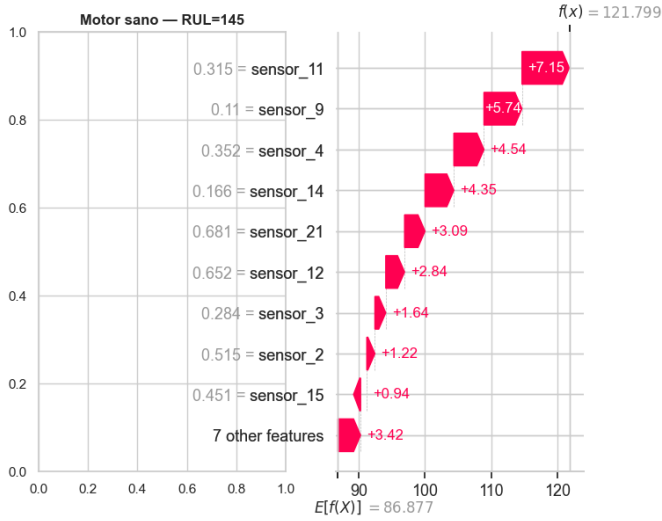


Fig. 8: Cross-dataset SHAP feature importance for XGBoost. Sensor_11 and sensor_4 maintain top positions across all four sub-datasets, confirming their role as universal degradation indicators.

TABLE IX: Top 5 Sensors by Mean Absolute SHAP Value — FD001

Sensor	Physical Description	XGBoost	RF
sensor_11	HPC outlet temperature	9.85	15.83
sensor_9	Total HPC pressure	7.14	8.91
sensor_4	LPC outlet temperature	5.88	4.73
sensor_14	Fan/bypass pressure	4.31	2.10
sensor_12	Corrected fan speed	4.11	3.60

The consistency of sensor_11 and sensor_4 across both model families validates their physical significance: HPC and LPC thermal degradation are the primary failure drivers in the C-MAPSS simulation. The SHAP dependence analysis (Figure 9) reveals non-linear threshold behavior in sensor_11, where its RUL impact increases exponentially above a critical value, consistent with accelerating compressor efficiency loss.

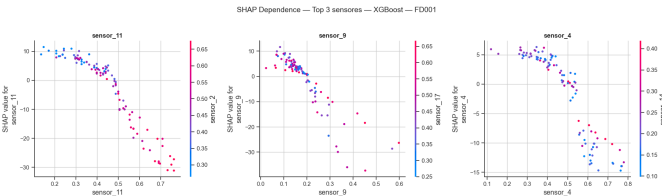


Fig. 9: SHAP dependence plots for the top 3 sensors. The non-linear threshold in sensor_11 corresponds to the onset of accelerating HPC degradation.

G. Hyperparameter Optimization Impact

Optuna revealed strong dataset-specificity in optimal configurations. FD002 favored compact architectures (hidden size = 64 for LSTM) with high dropout (0.45 for GRU), while FD001 benefited from larger models (hidden size = 192 for

Transformer, $d_{model} = 128$). FD003 and FD004 preferred smaller Transformer dimensions ($d_{model} = 32$), likely due to the increased noise from combined operating conditions and fault modes. These findings confirm that universal hyperparameter configurations are suboptimal: dataset-specific tuning is essential for robust deployment.

VI. DISCUSSION

A. Addressing Research Questions

RQ1 (ML vs. DL): DL ensembles outperform classical ML by 33–59% RMSE across all sub-datasets. This advantage derives from DL’s ability to process temporal sequences, exploit engineered rolling features, and learn non-linear degradation dynamics. Classical models are limited to single time-step snapshots with 16 base features.

RQ2 (Operational Complexity): Increasing complexity (FD001→FD004) degrades all models, but DL is significantly more resilient. Ensemble RMSE increases 45% from FD001 to FD004, versus 72% for classical ML. Operating condition normalization partially mitigates this degradation, but multi-fault dynamics remain challenging, particularly for TCN (RMSE 21.48 on FD004).

RQ3 (Feature Importance): SHAP consistently identifies sensor_11 (HPC outlet temperature) as the primary degradation indicator across all sub-datasets and both model families. This physical consistency validates the simulation’s design: HPC degradation is the dominant failure mode, and outlet temperature directly measures compressor efficiency loss. Sensor_4 (LPC outlet temperature) universally ranks second.

RQ4 (Binary Classification): AUC-ROC of 0.983 confirms that binary failure alerting is highly effective. The recall of 0.72 at $W = 30$ cycles suggests that adjusting the threshold to $W = 40$ –50 cycles would improve safety at acceptable precision cost. A two-stage strategy—binary alert followed by regression-based RUL estimation—offers a practical deployment framework.

B. Practical Implications

The ensemble approach (RMSE ≈ 12 cycles on FD001/FD002) provides sufficient accuracy for scheduling maintenance windows of 15–20 cycle duration. The SHAP results offer actionable guidance: continuous monitoring of sensor_11 and sensor_4 provides early warning capability for HPC degradation. For real-time deployment, the Transformer alone offers the best individual accuracy-to-cost trade-off ($R^2 \geq 0.83$ across all datasets), while the ensemble is preferred when inference latency permits.

C. Web Application

An interactive Streamlit application was developed and deployed on Render.com, providing: (1) real-time RUL prediction via uploaded sensor data, (2) model selection across ML, individual DL, and ensemble modes, (3) automatic comparison with ground truth RUL when available, including RMSE, R^2 , confusion matrix, and false-negative identification, (4) CSV export for integration with maintenance management systems.

VII. THREATS TO VALIDITY

Internal validity: DL models were evaluated on a validation split drawn from training data rather than the held-out test set, which lacks the multi-cycle temporal sequences required by sequence models. This introduces a potential discrepancy between reported and operational performance. Future work should develop methods to construct valid test sequences from the available operational snapshots.

External validity: All experiments use the simulated C-MAPSS dataset. Real-world turbofan degradation may exhibit different sensor noise characteristics, non-stationary fault progression, and environmental variability not captured in the simulation. Validation on field data, such as the N-CMAPSS dataset [17], is necessary before operational deployment.

Construct validity: The piece-wise linear RUL assumption with a fixed cap of 125 cycles is a simplification. Actual degradation trajectories are non-linear and engine-specific. The choice of $W = 30$ cycles for binary classification was based on prior literature but may require domain-expert calibration for specific fleet conditions.

Conclusion validity: The performance improvements reported are consistent across four sub-datasets and two model families, reducing the risk of spurious findings. Fixed random seeds (42) ensure full reproducibility. However, the 50-trial Optuna budget per combination may not fully explore the hyperparameter space, and results could improve with larger optimization budgets.

VIII. CONCLUSIONS

This paper presented a comprehensive comparative study of ML and DL approaches for turbofan RUL prediction using NASA C-MAPSS. Five key contributions emerge:

- 1) **Comprehensive benchmark:** Eight models across four algorithmic families evaluated on all four sub-datasets in a single integrated study.
- 2) **Automated optimization:** Optuna across 800 trials confirmed that dataset-specific tuning is essential; universal configurations are systematically suboptimal.
- 3) **Ensemble superiority:** Averaging LSTM, GRU, TCN, and Transformer predictions achieves $R^2 \geq 0.84$ on all sub-datasets, with 33–59% improvement over classical ML.
- 4) **Interpretability:** SHAP analysis identified sensor₁₁ and sensor₄ as universal degradation indicators, providing physically meaningful and actionable maintenance guidance.
- 5) **Deployment:** An interactive Streamlit application enables real-time RUL prediction, ensemble comparison, and ground-truth validation with full metrics.

Future work includes: (1) fault-mode-specific modeling for FD003/FD004, (2) uncertainty quantification via Monte Carlo Dropout or conformal prediction, (3) transfer learning across sub-datasets, (4) validation on the N-CMAPSS real-flight dataset [17], and (5) physics-informed neural network constraints to improve extrapolation beyond training conditions.

Technology Stack

Python 3.11, PyTorch 2.11 (CUDA 12.8, NVIDIA RTX 5060 Ti), Scikit-learn, XGBoost, Optuna, SHAP, Pandas, NumPy, Matplotlib, Seaborn, Plotly, Streamlit (deployed on Render.com). All experiments use seed 42. Source code available on GitHub.

REFERENCES

- [1] A. Saxena, K. Goebel, D. Simon, and N. Eklund, "Damage propagation modeling for aircraft engine run-to-failure simulation," in *Proc. PHM*, Denver, CO, 2008, pp. 1–9.
- [2] F. O. Heimes, "Recurrent neural networks for remaining useful life estimation," in *Proc. PHM*, Denver, CO, 2008, pp. 1–6.
- [3] E. Ramasso and A. Saxena, "Performance benchmarking and analysis of prognostic methods for CMAPSS datasets," *Int. J. PHM*, vol. 5, no. 2, pp. 1–15, 2014.
- [4] X. Li, Q. Ding, and J.-Q. Sun, "Remaining useful life estimation using deep convolution neural networks," *Reliability Eng. Syst. Safety*, vol. 172, pp. 1–11, 2018.
- [5] S. Zheng, K. Ristovski, A. Farahat, and C. Gupta, "Long short-term memory network for remaining useful life estimation," in *Proc. ICPHM*, Dallas, TX, 2017, pp. 88–95.
- [6] J. Wang et al., "Remaining useful life estimation using deep bidirectional LSTM," *Sensors*, vol. 19, no. 16, art. 3648, 2019.
- [7] A. L. Ellefsen et al., "Remaining useful life predictions using semi-supervised deep architecture," *Reliability Eng. Syst. Safety*, vol. 183, pp. 240–251, 2019.
- [8] J. Zhang et al., "Long short-term memory for machine remaining life prediction," *J. Manufacturing Systems*, vol. 48, pp. 78–86, 2019.
- [9] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. KDD*, San Francisco, CA, 2016, pp. 785–794.
- [10] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [11] K. Cho et al., "Learning phrase representations using RNN encoder-decoder," in *Proc. EMNLP*, Doha, Qatar, 2014, pp. 1724–1734.
- [12] C. Lea et al., "Temporal convolutional networks for action segmentation," in *Proc. CVPR*, Honolulu, HI, 2017, pp. 156–165.
- [13] A. Vaswani et al., "Attention is all you need," in *NeurIPS*, vol. 30, Long Beach, CA, 2017.
- [14] Y. Mo et al., "Remaining useful life estimation via Transformer encoder enhanced by a gated convolutional unit," *J. Intelligent Manufacturing*, vol. 32, no. 7, pp. 1997–2006, 2021.
- [15] H. Liu et al., "Remaining useful life prediction using a novel feature-attention-based end-to-end approach," *IEEE Trans. Industrial Informatics*, vol. 17, no. 2, pp. 1197–1207, 2020.
- [16] T. Akiba et al., "Optuna: A next-generation hyperparameter optimization framework," in *Proc. KDD*, Anchorage, AK, 2019, pp. 2623–2631.
- [17] J. P. Arias Chao et al., "Aircraft engine run-to-failure dataset under real flight conditions," *Data*, vol. 6, no. 1, art. 5, 2021.
- [18] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *NeurIPS*, vol. 30, Long Beach, CA, 2017, pp. 4765–4774.
- [19] L. Jayasinghe et al., "Temporal convolutional memory networks for remaining useful life estimation," in *Proc. ICIT*, Melbourne, 2019, pp. 915–920.
- [20] Y. Zhang et al., "LSTM for remaining useful life prediction of lithium-ion batteries," *IEEE Trans. Vehicular Technology*, vol. 67, no. 7, pp. 5695–5705, 2018.